UCIe Sideband RTL — Signal Direction Reference

Every port of every module: signal name, width, direction, exact source (From) and destination (To). Green = inputs, orange = outputs.

RX Architecture

```
SerDes  ──▶  SB_RX_FIFO  ──▶  ltsm_decoder  ──▶  LTSM
             ▲    |              ↓
          SB_RX_FSM         sb_error_handler
             ↓
        TIME_OUT_COUNTER
```

TX Architecture

```
LTSM  ──▶  ltsm_encoder  ──▶  SB_PACKET_ENCODER_MUX  ──▶  SB_TX_FIFO  ──▶  SB_TX_FSM_Modelling  ──▶
SB_CLOCK_CONTROLLER  ──▶  SerDes
                                  ▲
                           SB_PATTERN_GEN
SB_FSM controls ltsm_encoder   |   TIME_OUT_COUNTER  ──▶  LTSM  ──▶  SB_FSM
```

1. SB_CLOCK_CONTROLLER (clk_control.sv)

Generates gated 800 MHz sideband clock (TXCKSB) and per-packet timing pulses.

Signal	Width	Direction	From	To	Description
i_pll_clk	1	input	PLL	SB_CLOCK_CONTROLLER	800 MHz PLL clock
i_rst_n	1	input	System	SB_CLOCK_CONTROLLER	Active-low asynchronous reset
i_enable	1	input	SB_TX_FSM_Modelling	SB_CLOCK_CONTROLLER	Request to transmit one 64-bit packet
o_dead_time_done	1	output	SB_CLOCK_CONTROLLER	SB_TX_FSM_Modelling	HIGH for last cycle of 32-UI dead-time; ready
o_ser_done	1	output	SB_CLOCK_CONTROLLER	SB_TX_FSM_Modelling / SB_PATTERN_GEN	Single-cycle pulse when 64 bits shifted out
TXCKSB	1	output	SB_CLOCK_CONTROLLER	Sideband I/O pin	Gated sideband clock output to remote die

2. sb_error_handler (error_handler.sv)

i_pkt_valid from SB_RX_FSM (o_msg_valid). i_header / i_data / i_has_data from SB_RX_FIFO front entry. Sole owner of CP/DP parity checking.

Signal	Width	Direction	From	To	Description
clk	1	input	System	sb_error_handler	System clock
rst_n	1	input	System	sb_error_handler	Active-low reset
i_pkt_valid	1	input	SB_RX_FSM	sb_error_handler	o_msg_valid — packet committed to RX FIFO
i_header	64	input	SB_RX_FIFO	sb_error_handler	64-bit header flit from RX FIFO front entry
i_data	64	input	SB_RX_FIFO	sb_error_handler	64-bit data flit from RX FIFO front entry (ignor
i_has_data	1	input	SB_RX_FIFO	sb_error_handler	1 = front entry carries a data flit
o_cp_error	1	output	sb_error_handler	LTSM / Debug	CP (Control Parity) mismatch on current packe
o_dp_error	1	output	sb_error_handler	LTSM / Debug	DP (Data Parity) mismatch on current packet

3. ltsm_decoder (ltsm_decoder.sv)

o_cp_bit / o_dp_bit removed — parity owned by sb_error_handler. i_header / i_data from SB_RX_FIFO. o_msg_id feeds SB_RX_FSM for reset detection.

Signal	Width	Direction	From	To	Description
clk	1	input	System	ltsm_decoder	System clock

Signal	Width	Direction	From	To	Description
rst	1	input	System	ltsm_decoder	Active-high synchronous reset
i_pckt_valid	1	input	SB_RX_FSM	ltsm_decoder	o_msg_valid — complete packet committe
i_header	64	input	SB_RX_FIFO	ltsm_decoder	64-bit header flit from RX FIFO front entry
i_data	64	input	SB_RX_FIFO	ltsm_decoder	64-bit data flit from RX FIFO front entry (0 i
o_valid	1	output	ltsm_decoder	SB_RX_FSM / TIME_OUT_COUNTER / LTSM	1 = message found in SB_MSG_LUT
o_msg_id	sb_msg_id	output	ltsm_decoder	SB_RX_FSM / TIME_OUT_COUNTER / LTSM	Identified message enum value (11-bit)
o_msginfo	16	output	ltsm_decoder	LTSM	MsgInfo field [55:40] from header
o_data	64	output	ltsm_decoder	LTSM	Data flit forwarded to LTSM (valid when has
o_timeout_stop	1	output	ltsm_decoder	TIME_OUT_COUNTER	Any valid decoded message → stop 8-ms ti

4. ltsm_encoder (ltsm_encoder.sv)

o_ltsmpckt_valid / o_ltsm_header / o_ltsm_data all route to SB_PACKET_ENCODER_MUX ONLY — not directly to SB_FSM or SB_TX_FIFO.

Signal	Width	Direction	From	To	Description
rst	1	input	System	ltsm_encoder	Active-high synchronous reset
clk	1	input	System	ltsm_encoder	System clock
i_ltsm_valid	1	input	SB_FSM	ltsm_encoder	SB_FSM o_encoder_enable — trigger enc
i_msg_id	sb_msg_id	input	SB_FSM / LTSM	ltsm_encoder	Message ID to look up in SB_MSG_LUT
ltsm_data	64	input	LTSM	ltsm_encoder	Data payload (used when message has_d
ltsm_msginfo	16	input	LTSM	ltsm_encoder	MsgInfo field to embed in header [55:40]
o_ltsmpckt_valid	1	output	ltsm_encoder	SB_PACKET_ENCODER_MUX	Packet ready → MUX i_encoded_pkt_vali
o_ltsm_header	64	output	ltsm_encoder	SB_PACKET_ENCODER_MUX	64-bit header flit with CP/DP computed –
o_ltsm_data	64	output	ltsm_encoder	SB_PACKET_ENCODER_MUX	64-bit data flit → MUX data phase
o_timeout_start	1	output	ltsm_encoder	TIME_OUT_COUNTER	Pulse when packet built → start 8-ms tim
o_timeout_msg_id	sb_msg_id	output	ltsm_encoder	TIME_OUT_COUNTER	Msg ID sent — for exclusion check in time

5. SB_PACKET_ENCODER_MUX (pckt_mux.sv)

Releases encoded packet in two phases (header then data). Forwards o_final_has_data to TX FIFO.

Signal	Width	Direction	From	To	Description
i_pattern	64	input	SB_PATTERN_GEN	SB_PACKET_ENCODER_MUX	Start-pattern / PSTP priority word
i_encoded_packet	64	input	ltsm_encoder	SB_PACKET_ENCODER_MUX	Encoded 64-bit flit from ltsm_encoder (h
i_has_data	1	input	ltsm_encoder	SB_PACKET_ENCODER_MUX	1 = current packet has a data flit followir
i_pattern_valid	1	input	SB_PATTERN_GEN	SB_PACKET_ENCODER_MUX	Select i_pattern as output (highest priori
i_encoded_pkt_valid	1	input	ltsm_encoder	SB_PACKET_ENCODER_MUX	Select i_encoded_packet as output (ltsm
o_final_packet	64	output	SB_PACKET_ENCODER_MUX	SB_TX_FIFO	Muxed 64-bit word to TX FIFO; 0 when id
o_final_valid	1	output	SB_PACKET_ENCODER_MUX	SB_TX_FIFO	o_final_packet is valid this cycle
o_final_has_data	1	output	SB_PACKET_ENCODER_MUX	SB_TX_FIFO	Forwarded has_data flag so TX FIFO know

6. SB_RX_FIFO (rx_fifo.sv)

Mirrors TX FIFO. Stores header + data + has_data per entry. Exposes o_is_pattern (combinational) so SB_RX_FSM can do pattern detection without ever reading a 64-bit word.

Signal	Width	Direction	From	To	Description
i_clk	1	input	System	SB_RX_FIFO	Clock
i_rst_n	1	input	System	SB_RX_FIFO	Active-low asynchronous reset
i_rx_header_valid	1	input	SB_RX_FSM	SB_RX_FIFO	Phase-1: latch incoming word into assembly r
i_rx_header	64	input	SerDes	SB_RX_FIFO	64-bit received header flit from deserialiser
i_rx_data_valid	1	input	SB_RX_FSM	SB_RX_FIFO	Phase-2: latch incoming word into assembly r
i_rx_data	64	input	SerDes	SB_RX_FIFO	64-bit received data flit from deserialiser
i_has_data	1	input	SB_RX_FSM	SB_RX_FIFO	1 = this packet carries a data flit; stored with t
i_write_commit	1	input	SB_RX_FSM	SB_RX_FIFO	Pulse: push fully assembled header+data entr
i_read_enable	1	input	SB_RX_FSM	SB_RX_FIFO	Pop front entry — decoder ready to consume
o_header	64	output	SB_RX_FIFO	ltsm_decoder / sb_error_handler	Header flit of front entry
o_data	64	output	SB_RX_FIFO	ltsm_decoder / sb_error_handler	Data flit of front entry
o_has_data	1	output	SB_RX_FIFO	ltsm_decoder / sb_error_handler / SB_RX_FSM	1 = front entry has a data flit
o_is_pattern	1	output	SB_RX_FIFO	SB_RX_FSM	Combinational: assembly-register word == SB_ pattern detection without inspecting raw data
o_valid	1	output	SB_RX_FIFO	SB_RX_FSM	Entry ready for decoder (!o_empty)
o_empty	1	output	SB_RX_FIFO	SB_RX_FSM	FIFO empty flag
o_full	1	output	SB_RX_FIFO	Debug	FIFO full flag
o_overflow	1	output	SB_RX_FIFO	Debug	Write attempted while full (sticky pulse)

7. SB_RX_FSM (rx_fsm.sv)

CONTROL ONLY — no header/data ports, no parity logic. Receives only i_de_ser_done pulse from SerDes. Uses i_fifo_is_pattern for pattern detection, i_fifo_has_data for sequencing, i_msg_id for reset detection. Parity (CP/DP) handled exclusively by sb_error_handler.

Signal	Width	Direction	From	To	Description
i_clk	1	input	System	SB_RX_FSM	Clock
i_rst_n	1	input	System	SB_RX_FSM	Active-low asynchronous reset
i_de_ser_done	1	input	SerDes	SB_RX_FSM	Pulse: new 64-bit word captured. Raw word receives pulse only
i_fifo_is_pattern	1	input	SB_RX_FIFO	SB_RX_FSM	HIGH when FIFO assembly register == SB_ST detection
i_fifo_has_data	1	input	SB_RX_FIFO	SB_RX_FSM	Authoritative has_data flag for current entry phase
i_fifo_valid	1	input	SB_RX_FIFO	SB_RX_FSM	RX FIFO has a committed entry ready for dec
i_msg_id	11	input	ltsm_decoder	SB_RX_FSM	Decoded message ID — FSM checks for rese
i_msg_id_valid	1	input	ltsm_decoder	SB_RX_FSM	ltsm_decoder o_valid — i_msg_id is valid thi
o_rx_header_valid	1	output	SB_RX_FSM	SB_RX_FIFO	Phase-1: instruct RX FIFO to latch current wo
o_rx_data_valid	1	output	SB_RX_FSM	SB_RX_FIFO	Phase-2: instruct RX FIFO to latch current wo
o_write_commit	1	output	SB_RX_FSM	SB_RX_FIFO	Pulse: commit assembled header+data entry
o_fifo_read_enable	1	output	SB_RX_FSM	SB_RX_FIFO	Pop front entry — notifies decoder that a co
o_rx_sb_start_pattern	1	output	SB_RX_FSM	TX path	Request TX side to begin sending start-patte

Signal	Width	Direction	From	To	Description
o_rx_sb_pattern_samp_done	1	output	SB_RX_FSM	TIME_OUT_COUNTER / SB_PATTERN_GEN	128 UI start-pattern locked
o_msg_valid	1	output	SB_RX_FSM	ltsm_decoder / sb_error_handler	Complete packet in RX FIFO and ready for pr
o_de_ser_done_sampled	1	output	SB_RX_FSM	SerDes	Registered ack back to deserialiser

8. TIME_OUT_COUNTER (timeout_counter.sv)

o_time_out → LTSM ONLY. LTSM propagates to SB_FSM / error path as needed.

Signal	Width	Direction	From	To	Description
i_clk	1	input	System	TIME_OUT_COUNTER	System clock (800 MHz)
i_rst_n	1	input	System	TIME_OUT_COUNTER	Active-low asynchronous reset
i_start	1	input	ltsm_encoder	TIME_OUT_COUNTER	Start counting — TX message sent by enco
i_stop	1	input	ltsm_decoder	TIME_OUT_COUNTER	Any valid RX message decoded → stop co
i_msg_id	sb_msg_id	input	ltsm_encoder	TIME_OUT_COUNTER	TX msg_id — exclude non-timed states fro
i_rx_msg_id	sb_msg_id	input	ltsm_decoder	TIME_OUT_COUNTER	RX msg_id — stall condition 2 check
i_rx_msginfo	16	input	ltsm_decoder	TIME_OUT_COUNTER	RX MsgInfo — stall condition 1 (==16hFFF
i_start_pattern_req	1	input	SB_PATTERN_GEN / LTSM	TIME_OUT_COUNTER	SBINIT pattern phase entered → arm the
i_rx_sb_pattern_samp_done	1	input	SB_RX_FSM	TIME_OUT_COUNTER	Partner pattern locked → stop counter
i_pattern_time_out	1	input	SB_PATTERN_GEN	TIME_OUT_COUNTER	TX-side 8 ms expired — passed through to
o_time_out	1	output	TIME_OUT_COUNTER	LTSM	8 ms timeout expired. LTSM ONLY — LTSM

9. SB_TX_FIFO (tx_fifo.sv)

Write side from SB_PACKET_ENCODER_MUX. Read side to SB_TX_FSM_Modelling.

Signal	Width	Direction	From	To	Description
i_clk	1	input	System	SB_TX_FIFO	Clock
i_rst_n	1	input	System	SB_TX_FIFO	Active-low asynchronous reset
i_write_enable	1	input	SB_PACKET_ENCODER_MUX	SB_TX_FIFO	Push encoded packet into FIFO (o_final_val
i_header	64	input	SB_PACKET_ENCODER_MUX	SB_TX_FIFO	64-bit header flit (o_final_packet, header p
i_data	64	input	SB_PACKET_ENCODER_MUX	SB_TX_FIFO	64-bit data flit (o_final_packet, data phase;
i_has_data	1	input	SB_PACKET_ENCODER_MUX	SB_TX_FIFO	1 = entry carries a data flit (o_final_has_da
i_read_enable	1	input	SB_TX_FSM_Modelling	SB_TX_FIFO	Pop front entry
o_header	64	output	SB_TX_FIFO	SB_TX_FSM_Modelling	Header of front entry
o_data	64	output	SB_TX_FIFO	SB_TX_FSM_Modelling	Data flit of front entry
o_has_data	1	output	SB_TX_FIFO	SB_TX_FSM_Modelling	1 = front entry has data flit; FSM sequences
o_empty	1	output	SB_TX_FIFO	SB_TX_FSM_Modelling	FIFO empty flag
o_full	1	output	SB_TX_FIFO	Debug	FIFO full flag
o_overflow	1	output	SB_TX_FIFO	Debug	Write attempted while full
o_ser_done_sampled	1	output	SB_TX_FIFO	SB_TX_FSM_Modelling	Registered copy of i_read_enable — read a

10. SB_FSM (tx_fsm.sv)

i_time_out received from LTSM (which relays TIME_OUT_COUNTER o_time_out). Not wired directly from the counter.

Signal	Width	Direction	From	To	Description
i_clk	1	input	System	SB_FSM	Clock
i_rst_n	1	input	System	SB_FSM	Active-low asynchronous reset
i_start_pattern_req	1	input	LTSM / SB_PATTERN_GEN	SB_FSM	SBINIT: begin start-pattern TX
i_msg_valid	1	input	LTSM	SB_FSM	New SB message ready to encode
i_ltsm_pckt_valid	1	input	ltsm_encoder	SB_FSM	Encoder output valid — packet fully built (also
i_sb_ltsm_resp_rcvd	1	input	ltsm_decoder / SB_RX_FSM	SB_FSM	Genuine LTSM RESP received from remote die
i_time_out	1	input	LTSM	SB_FSM	8 ms timeout relayed by LTSM (TIME_OUT_CO
i_start_pattern_done	1	input	SB_PATTERN_GEN	SB_FSM	Pattern generator finished
o_encoder_enable	1	output	SB_FSM	ltsm_encoder	One-cycle pulse to trigger ltsm_encoder (→ i_
o_busy	1	output	SB_FSM	LTSM	FSM is actively processing (not in IDLE)
o_rsp_timeout	1	output	SB_FSM	LTSM	Latched timeout flag → LTSM drives state to T

11. SB_TX_FSM_Modelling (tx_fsm_modeling.sv)

Uses i_has_data from TX FIFO to sequence header phase then data flit phase.

Signal	Width	Direction	From	To	Description
i_clk	1	input	System	SB_TX_FSM_Modelling	800 MHz SB clock
i_rst_n	1	input	System	SB_TX_FSM_Modelling	Active-low asynchronous reset
i_ser_done	1	input	SB_CLOCK_CONTROLLER	SB_TX_FSM_Modelling	64 bits shifted out (o_ser_done)
i_empty	1	input	SB_TX_FIFO	SB_TX_FSM_Modelling	TX FIFO empty flag
i_dead_time_done	1	input	SB_CLOCK_CONTROLLER	SB_TX_FSM_Modelling	32-UI dead-time complete (o_dead_time_don
i_read_enable_sampled	1	input	SB_TX_FIFO	SB_TX_FSM_Modelling	TX FIFO read ack (o_ser_done_sampled)
i_has_data	1	input	SB_TX_FIFO	SB_TX_FSM_Modelling	Front entry has a data flit → sequence heade
o_read_enable	1	output	SB_TX_FSM_Modelling	SB_TX_FIFO	Pop header from TX FIFO
o_read_data_enable	1	output	SB_TX_FSM_Modelling	SB_TX_FIFO	Pop data flit from TX FIFO (second read, has_
o_clk_en	1	output	SB_TX_FSM_Modelling	SB_CLOCK_CONTROLLER	Enable → i_enable; triggers one 64-bit serialis

12. SB_PATTERN_GEN (tx_pattern_generator.sv)

Outputs to SB_PACKET_ENCODER_MUX (highest priority). o_pattern_time_out feeds TIME_OUT_COUNTER.

Signal	Width	Direction	From	To	Description
i_clk	1	input	System	SB_PATTERN_GEN	800 MHz SB clock
i_rst_n	1	input	System	SB_PATTERN_GEN	Active-low asynchronous reset
i_start_pattern_req	1	input	LTSM (SBINIT)	SB_PATTERN_GEN	Assert when LTSM enters SBINIT pattern p
i_rx_sb_pattern_samp_done	1	input	SB_RX_FSM	SB_PATTERN_GEN	Partner detected our pattern (128 UI lock
i_ser_done	1	input	SB_CLOCK_CONTROLLER	SB_PATTERN_GEN	One 64-bit word serialised → advance iter
o_start_pattern_done	1	output	SB_PATTERN_GEN	SB_FSM / LTSM	Pattern phase complete (4 extra iterations
o_pattern_time_out	1	output	SB_PATTERN_GEN	TIME_OUT_COUNTER	8-ms TX-side timeout → TIME_OUT_COUN
o_pattern	64	output	SB_PATTERN_GEN	SB_PACKET_ENCODER_MUX	Pattern word 64h5555_5555_5555_5555
o_pattern_valid	1	output	SB_PATTERN_GEN	SB_PACKET_ENCODER_MUX	o_pattern valid this cycle → MUX i_patter